

1. Veiledet læring

1.1 Data

Variabler (statistikk) eller features (maskinlæring) er samme greien.

Kvalitative eller kvantitative variabler:

- Kvalitative representerer kategorier.
 - Sjanger, aldersgruppe, type frukt ...
- Kvantitative representerer numeriske verdier:
 - Inntekt, alder, antall lyttere, areal, nedbørsmengde, temperatur...

Features benevnes x_n

Targets benevnes y_n

Indekserer med (i) som slik: $x^{(i)}$

Eksempel Titanic datasett:

Table 3: Titanic-datasettet etter at vi har valgt ut hvilke variable vi vil beholde, og innført notasjon.

	x_1	x_2	x_3	y
$\mathbf{x}^{(1)}$	3	male	22	0
$\mathbf{x}^{(2)}$	1	female	38	1
$\mathbf{x}^{(3)}$	3	female	26	1
$\mathbf{x}^{(4)}$	1	female	35	1
$\mathbf{x}^{(5)}$	3	male	35	0
$\mathbf{x}^{(6)}$	3	male		0
$\mathbf{x}^{(7)}$	1	male	54	0
$\mathbf{x}^{(8)}$	3	male	2	0
$\mathbf{x}^{(9)}$	2	male	27	1

Vi vil fjerne hullede data, encode kvalitative variabler og endre størrelsesorden på numeriske verdier. Deretter kan vi se på dataene!

1.2 Logistisk regresjon

Enkleste modellen er lineær regresjon:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}\mathbf{x} + b$$

Der $\mathbf{w}$ er vektene og b er bias.

Kan gjøres om til sansynlighetsfordeling (mellom 0 og 1) ved å sette inn for [#Sigmoid-funksjonen](#) (også kjent som [#aktiveringsfunksjon](#))

$$\sigma(z) = \frac{1}{1 + e^{-z}} \implies y_{pred} = \frac{1}{1 + e^{-(\sum_{i=1}^n w_i x_i + b)}}$$

1.2.1 Trening og tapsfunksjon

Funksjonen som brukes til å beregne målopnnåelse kalles for `#tapsfunksjon`. Forteller hvor nærme modellens prediksjon er den riktige verdien, eller target:

$$\mathcal{L}(y_{pred}, y)$$

Den tar inn modellprediksjonene y_{pred} og `#targets` y . **NB!** må være deriverbar (se 1.2.2 Gradient descent)

[Eksempel link](#)

Eksempel på funksjon for klassifisering av binær target:

$$L(y_{pred}, y) = y_{pred}^y (1 - y_{pred})^{1-y}$$
$$\implies L(\mathbf{w}, b|x, y) = \sigma(\mathbf{w}\mathbf{x} + b)^y (1 - \sigma(\mathbf{w}\mathbf{x} + b))^{(1-y)}$$

Vi vil finne en `#likelihood` og ikke en sannsynlighet fordi vi **ikke** har lyst til å endre dataene slik at sannsynligheten for target endrer seg. men heller endre modellen slik at den estimerte sannsynligheten passer med den virkelige (target).

Vi ønsker å ha likelihood for å estimere riktig være maksimal, så vi kan legge til et minustegn på tapsfunksjonen for å minimere denne. Vi bruker gjerne å ta logaritmen av av likelihood for å få **log-likelihood** (Minimere tap == maksimere Likelihood). Dette kalles for `#cross-entropy-loss`.

Eksempel på tapsfunksjon: `#binary_cross-entropy-loss`

Vektet tapsfunksjon: Maksimere likelihood for riktig kategorisering av dataene

$$\mathcal{L}(y_{pred}, y) = -\omega_1 y \ln y_{pred} - \omega_0 (1 - y) \ln(1 - y_{pred})$$

Der ω_0 og ω_1 er vekter for klassene, se [Vektet tapsfunksjon](#).

y er target og vi kan bruke y_{pred} fra [formelen her](#).

Vi kan nå skrive om modellen definert av parametrene $\mathbf{w}$ og b for å finne parametrene som minimerer tapet i gjennomsnitt over alle datapunktene (x). Dette kaller vi for å **trene modellen**. Vi får da at vi skal maksimere log-likelihooden for parametrene som vi definerer:

$$\theta = (\mathbf{w}, b) \implies \hat{\theta} = \arg_{\theta} \min \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

Detta kalles **conditional maximum likelihood estimation**, og kan løses med bruk av [gradient descent](#).

1.2.2 Gradient descent

Når vi ser på funksjonen vår beveger vi oss i rommet spent ut av verdiene θ , fire verdier gir et fire dimensjonalt rom. Vi vil finne minimumspunktet til funksjonen uten at vi

kjenner formen til den. **NB!** så lenge tapsfunksjonen er konveks, så vil vi alltid kunne finne lokalt minimum med gradienten.

I starten av treningen har alle parametrene en tilfeldig verdi, og vi oppdaterer verdiene iterativt, gjennom flere steg. Gradienten til tapsfunksjonen peker i retning av økende tap, og siden vi ønsker å minke tapet må vi bevege oss i motsatt retning. Dette representerer vi gjennom et minustegn, og regelen for parameteroppdateringen kan skrives som følger:

$$\theta^{t+1} = \theta^t - \eta \frac{\partial}{\partial \theta} \mathcal{L}(f(\mathbf{x}; \theta), y)$$

Der η er `#læringsraten`, som bestemmer hvor mye hvert steg t skal korrigere til parameterverdien. **NB!** η eller læringsraten er en parameter for å justere treningen til modellen, men er **ikke** en del av modellen! Dette kaller vi for en `#hyperparameter`, som definerer læringsprosessen, men ikke modellen. Den kan vi leke med/justere mye for å optimalisere, men generelt ønsker små steg for å ikke hoppe over minimumspunktet men også ikke fordi da vil modellen bruke lang tid på å læres.

Framgangsmåten vil da bli å finne gradienten av tapsfunksjonen, med hensyn til vær av parametrene. Eksempel med titanic modellen:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} [-y \ln \sigma(\mathbf{w}\mathbf{x} + b) - (1 - y) \ln (1 - \sigma(\mathbf{w}\mathbf{x} + b))] \\ &= -y \frac{1}{\sigma(\cdot)} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) + (1 - y) \frac{1}{1 - \sigma(\cdot)} \frac{\partial}{\partial w_j} (1 - \sigma(\mathbf{w}\mathbf{x} + b)) \\ &= \frac{-y(1 - \sigma(\cdot)) + (1 - y)\sigma(\cdot)}{\sigma(\cdot)(1 - \sigma(\cdot))} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) \\ &= \frac{\sigma(\cdot) - y}{\sigma(\cdot)(1 - \sigma(\cdot))} \sigma(\cdot)(1 - \sigma(\cdot)) \frac{\partial}{\partial w_j} (\mathbf{w}\mathbf{x} + b) \\ &= (\sigma(\mathbf{w}\mathbf{x} + b) - y) x_j \\ &= (y_{\text{pred}} - y) x_j \\ \frac{\partial \mathcal{L}}{\partial b} &= \dots \\ &= (\sigma(\mathbf{w}\mathbf{x} + b) - y) \frac{\partial}{\partial b} (\mathbf{w}\mathbf{x} + b) \\ &= (y_{\text{pred}} - y) . \end{aligned}$$

1.2.3 Trening

En måte å strukturere `#læringsalgoritme` på:

```

class LogisticRegression:
    def __init__(self, learning_rate=0.1, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights,
        self.bias = None, None
        self.losses, self.train_accuracies = [], []

    def sigmoid_function(self, x):

    def _compute_loss(self, y, y_pred):

    def compute_gradients(self, x, y, y_pred):

    def update_parameters(self, grad_w, grad_b):

    def accuracy(true_values, predictions):
        return np.mean(true_values == predictions)

    def fit(self, x, y):
        self.weights = np.zeros(x.shape[1]) #x.shape = datapunkter, features
        self.bias = 0

        # Gradient Descent
        for _ in range(self.epochs):
            lin_model = np.matmul(self.weights, x.transpose()) + self.bias
            y_pred = self._sigmoid(lin_model)
            grad_w, grad_b = self.compute_gradients(x, y, y_pred)
            self.update_parameters(grad_w, grad_b)

            loss = self._compute_loss(y, y_pred)
            pred_to_class = [1 if _y > 0.5 else 0 for _y in y_pred]
            self.train_accuracies.append(accuracy(y, pred_to_class))
            self.losses.append(loss)

    def predict(self, x):
        lin_model = np.matmul(x, self.weights) + self.bias
        y_pred = self._sigmoid(lin_model)
        return [1 if _y > 0.5 else 0 for _y in y_pred]

```



Når skal bruke koden, bør en **ikke** bruke hele datasettet til å trene modellen. Del dataen opp i `#testdata` og `#treningsdata`. Treningsdataene brukes til parametertilpasning, mens testdataene ikke røres før modellen er ferdig tilpasset, og brukes for å evaluere modellen før denne tas i bruk. Bruk gjerne `train_test_split` fra biblioteket `scikit-learn`.

Eksempel på kjøring av kode:

```
# Training
train_epochs = 30

# Initialize and train the model
log_reg = LogisticRegression(learning_rate=0.01, epochs=train_epochs)
log_reg.fit(X_train, y_train)

# Make predictions
predictions = log_reg.predict(X_test)
```



1.2.4 Evaluering

Lurt å plotte tapet (loss) mot treffsikkerhet (accuracy) for å se treningsprosedyren er god.

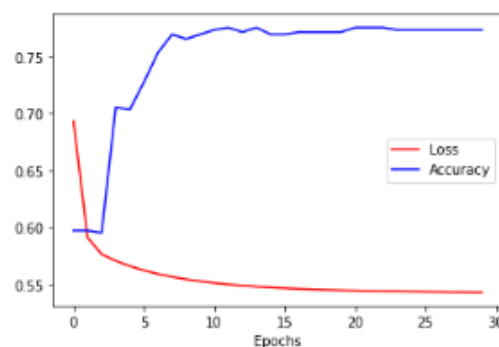


Figure 4: Eksempel på plott som viser tap (rød) og treffsikkerhet (blå) over epokene i en treningsprosedyre.

Mange metrikker baseres på `#confusion_matrix` som brukes ofte for å evaluere klassifiseringsmodeller.

		Predicted	
		Negative (PN)	Positive (PP)
True	Total (P+N)		
	Negative (N)	True negative	False positive
	Positive (P)	False negative	True positive

Figure 5: Confusion matrix for binær klassifisering.

Dette kan vi bruke til å regne ut flere metrikker som i tabellen under:

Table 4: Et utvalg metrikker vi kan beregne basert på verdiene i confusion matrix.

Betegnelse	Uttrykk	Alternative navn
True Positive Rate (TPR)	$\frac{TP}{P} = \frac{TP}{TP+FN}$	Sensitivitet, recall, sannsynlighet for deteksjon
True Negative Rate (TNR)	$\frac{TN}{N} = \frac{TN}{TN+FP}$	Spesifisitet
False Positive Rate (FPR)	$\frac{FP}{N} = \frac{FP}{FP+TN}$	Sannsynlighet for falsk alarm, (1 - spesifisitet)
Positive Predictive Value (PPV)	$\frac{TP}{TP+FP}$	Precision

1.2.5 Klassifiseringsterskel

Enkel måte å justere på klassifiseringsterskelen er å endre linjen i `fit` og `predict` funksjonene:

```
[1 if _y > K else 0 for _y in y_pred]
```

Der K er klassifiseringsterskelen. Hvis en vil være sikker på at modellens predikasjon er riktig, vil en velge en høyere verdi for klassifiseringsterskelen som 0.7.

Det er vanlig å plote **FPR** (False Positive Rate) og **TPR** (True Positive Rate) som funksjon av klassifiseringsterskelen. Dette gir oss **Receiver Operating Characteristic** (`#ROC`)-kurven. Området under kurven til ROC kaller vi for `#AUC` (area under curve) som vi ønsker skal være større enn 0.5 (tilsvarer tilfeldig gjetting). ROC AUC representerer sannsynligheten for at modellen vil predikere en høyere verdi for et tilfeldig valgt positivt datapunkt enn for et tilfeldig valgt negativt datapunkt.

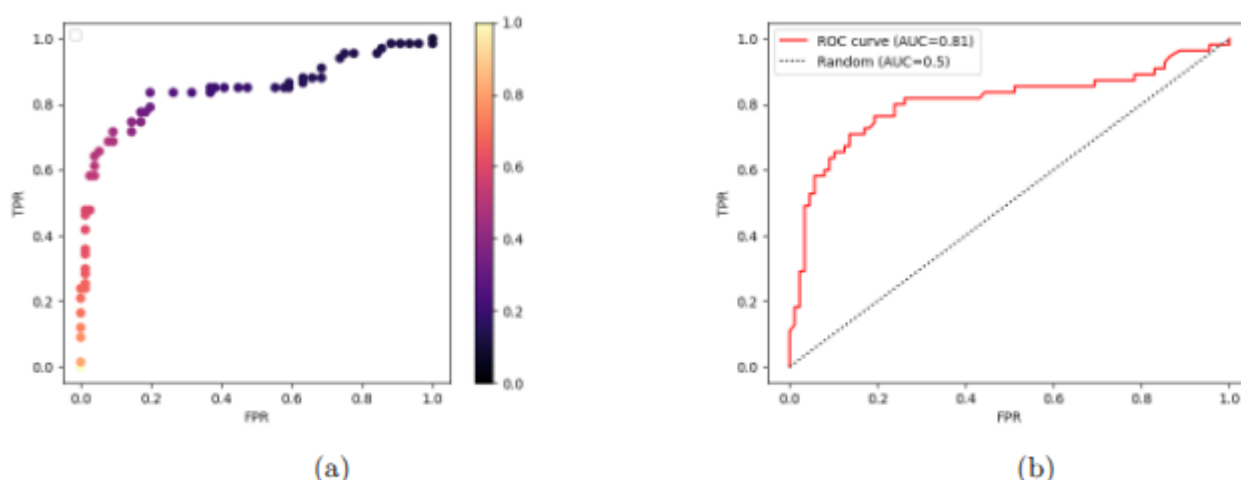


Figure 6: ROC-kurven, som viser FPR og TPR for ulike klassifiseringsterskler.

Ingen tradeof mellom TPR og FPR, men alltid en tradeoff med metrikkene en ønsker å ha høy:

- En annen vanlig kurve å studere er den som viser precision (TPR) og recall (PPV).

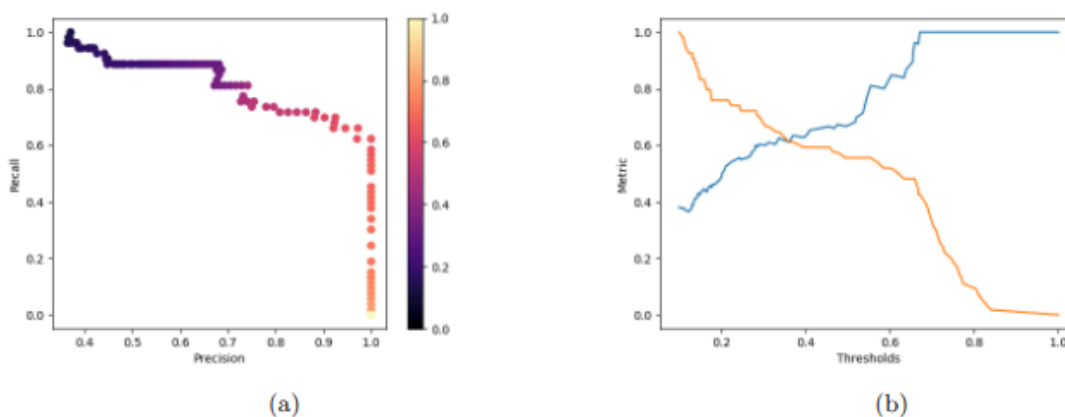


Figure 7: (a) Precision-recall-kurven, og (b) henholdsvis precision- og recall-kurven, for ulike klassifiseringsterskler.

Presisjon: Angir andel korrekt predikert positive per predikert positiv.

- Høy presisjon svarer til lavt relativt antall falske alarmer.
- Viktig om falske alarmer er dyrt, som å rykke ut til brann.

Recall: Angir andel korrekt predikert positive per totalt positive.

- Høy recall svarer til lavt antall missed cases.
- Viktig om kritisk å ikke treffe på en case, som med detektore kreft.

1.2.6 Kort om entropi og cross entropy loss

I informasjonsteori er entropi (også kalt Shannon entropy) en måte å kvantisere usikkerheten eller mengden informasjon tilgjengelig i en variabels mulige tilstander:

$$H(p) = - \sum_i p(x_i) \log p(x_i). \quad (28)$$

I tilfellet med Titanic-dataene representerer $p(x_i)$ her sannsynligheten for **Survived**. Kryss-entropien (cross entropy) representerer da forskjellen mellom to ulike sannsynlighetsfordelinger p og q :

$$H(p, q) = - \sum_i p(x_i) \log q(x_i). \quad (29)$$

Hvis vi sammenligner dette med `#tapsfunksjon` og `#cross-entropy-loss`, ser vi at den ene sannsynlighetsfordelingen representerer labels i datasettet, mens den andre representerer modellens prediksjoner. Tapsfunksjonen forteller oss altså om forskjellen mellom de to fordelingene, og siden målet vårt er å lage en klassifiseringsmodell, ønsker vi at forskjellen mellom de to fordelingene er minst mulig. Vi kan generalisere uttrykket over til:

$$\mathcal{L}(\hat{y}, y) = - \sum_i y_i \log(\hat{y}_i),$$

hvor summen går over alle klassene i .

1.2.7 Bayes' teorem

Vi kan bruke Bayes' teorem til å regne:

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}$$

Vi får da at:

- $p(C_k)$ er *a priori* sannsynligheten for klasse, altså fordelingen av klassene i treningsdataene. Denne omtales som prior, fordi det er sannsynligheten for **klassetilhørighet** vi kan estimere uten å vite noe om det aktuelle datapunktet (legg merke til at x ikke forekommer i dette leddet).
- $p(x)$ er fordelingen av selve dataene, uavhengig av klasse (legg merke til at C_k ikke forekommer i dette leddet). Dette er den faktiske sannsynlighetsfordelingen av alle

feature-verdiene for alle dataene, og den har like mange dimensjoner som datasettet har features. Dette leddet omtales som evidens.

- $p(x|C_k)$ er den betingede sannsynlighetsfordelingen av dataene gitt klassen, og det er en likelihood. Det er altså en egen sannsynlighetsfordeling – med like mange dimensjoner som datasettet har features – for hver klasse.

Vi kan regne $p(C_k)$, og for titanic casen kan vi se at $p(x)$ vil være vanskelig å bergne, men at den bidrar like mye til begge klassene så vi kan se vekk i fra den og få at:

$$p(C_k) \propto p(x|C_k)p(C_k)$$

Hvis vi antar at verdiene til featuresene er uavhengige av hverandre for alle features, så kan vi skrive dette om til Naiv Bayes:

$$p(C_k|x) \propto p(C_k) \prod_{i=1}^n p(x_i|C_k)$$

Som gir at den tilsvarende sannsynligheten for klassetilhørighet:

$$\hat{y} = \arg_{k \in [1, \dots, n]} \max p(C_k) \prod_{i=1}^n p(x_i|C_k)$$

1.2.8 Dimensjonsforbannelsen

Om vi ønsker å estimere en d -dimensjonal fordeling, er antallet datapunkt som må samles inn avhengig av d . Hvis vi tenker oss at vi estimerer fordelingen ved hjelp av et histogram med 10 bins (per dimensjon) hvor hver bin har 10 datapunkter (100 datapunkter per dimensjon). Et mye brukt datasett [#MNIST](#) har 784 dimensjoner, og det finnes ca 10^{82} atomer i universet.

Dimensjoner d	Bins	Datapunkter
1	10	10^2
3	10^3	10^4
10	10^{10}	10^{11}
784	10^{784}	10^{785}

1.2.9 Trening på ubalansert data

Om vi har mye flere datapunkter for forskjellige klasser, ender vi opp med å få **underrepresentert** og **overrepresentert** klasser. Hvis forskjellen mellom klasser blir for stor, vil modellen belønnes om den bare predikerer at alle datapunktene tilhører den overrepresenterte klassen som gir en dårlig modell. For å justere skjevfordelingen mellom klassene kan vi endre på klassifiseringsterskelen som i uttrykket:

$$y_pred = [1 \text{ if } _y > K=0.5 \text{ else } 0 \text{ for } _y \text{ in } y_pred]$$

fra 0.5 til en verdi som gir modellen høyere verdi på de andre metrikkene. Kan bruke precision-recall plottet som et utgangspunkt. Å maksimere en metrikk kommer på bekostning av andre metrikker og å justere terskelen forbedrer **ikke** modellen: det endrer kun hvordan vi forholder oss til modellens prediksjoner. Vi kan også endre dataene eller tapsfunksjonen.

Resampling

Undersampling:

Her trekker vi like mange datapunkter fra den overrepresenterte klassen som vi har tilgjengelig i den underrepresenterte klassen. Da ender vi opp med et datasett bestående av like mange datapunkter fra hver klasse, men potensielt veldig få datapunkter totalt. Dette kan føre til at modellen som trenes på dataene undertilpasser (underfit).

Oversampling:

Her kopierer vi instanser fra den underrepresenterte klassen, inntil vi har like mange datapunkter fra den underrepresenterte som fra den overrepresenterte klassen. Da ender vi også opp med like mange datapunkter fra hver klasse, men potensielt mange duplikater fra den underrepresenterte klassen. Dette kan føre til at modellen som trenes på dataene overtilpasser (overfit)

Vektet tapsfunksjon

Vi kan også fortelle modellen hvilken av klassene som er ekstra viktig ved å straffe feilprediksjoner (høyere tap) på den viktige klassen, relativt til de andre klassene. F.eks med `#binary_cross-entropy-loss`. Vi ser at for de to leddene, bidrar kun det første leddet til tapet når $y=1$ og det andre leddet når $y=0$. Ved å sette inn vektene w_1 & w_0 for henholdsvis første og andre leddet kan vi gjøre at feilprediksjoner for den ene vil gi ut høyere tap enn den andre.

1.3 Beslutningstrær

1.3.1 Noder

`#Root-node` eller rotnoden er starten på beslutningstreet av dataene med alle featuresene som er splittet i en trestruktur.

Under har vi beslutningsnoder (decision nodes) som begge har kriterier for å splitte dataene (splitting criteria). Alle noder som splitter dataene er enten rotnoden eller beslutningsnoder.

Nederst i treet er løvnodene (leaf nodes) som ikke splitter dataene, som inneholder predikert verdi for datainstansen som ble sendt gjennom treet.

Beslutningstrær består av trestumper (tree stumps) som igjen består av en rotnode og n løvhoder, for n mulige utfall.

Treningsdataene brukes for å finne ut hvilke trestumper (og tilhørende beslutningskriterier) som bør settes sammen for å lage treet. Når vi bygger beslutningstrær ønsker vi alltid å velge det splitt-kriteriet som lar oss ta beslutningen tidligst mulig, altså reduserer usikkerheten mest mulig. Redusert usikkerhet er endringen i usikkerhet etter sammenliknet med før splitt. I hovedsak brukes følgende tre metrikker for å måle hvor mye et splitt-kriterium (feature og verdi) reduserer usikkerheten:

- Log loss (Se `#tapsfunksjon` og `#cross-entropy-loss`)
- Gini impurity
- Entropi

1.3.2 Gini impurity

Gini-urenheten er et tall i $[0, 0.5]$ som angir sannsynligheten for at et nytt, tilfeldig datapunkt feilklassifiseres hvis det gis et tilfeldig label i henhold til klassedistribusjonen i datasettet. Gitt et datasett D bestående av datapunkter fra k klasser, med sannsynlighet p_i for at en instans tilhører klassen i ved en gitt node, er datasettets Gini-urenhet:

$$\text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2$$

Intuitivt: Du har en pose med kuler i ulike farger, hvor farge representerer klasse. Gini-urenheten måler hvor sannsynlig det er at du gjetter feil farge på en tilfeldig trukket kule, hvis du gjetter at kulens farge følger distribusjonen av farger i posen.

Lav Gini-urenhet representerer scenariet der de fleste kulene har samme farge, slik at det er lav sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha lav urenhet.

Høy Gini-urenhet representerer motsatt scenario, der klassene er forholdsvis likt representert, og det er høy sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha høy urenhet.

Hvis dataset D splittes på feature f til to subsett D_1 og D_2 med henholdsvis n_1 og n_2 datapunkter, har vi

$$\text{Gini}_f(D) = \frac{n_1}{n} \text{Gini}(D_1) + \frac{n_2}{n} \text{Gini}(D_2)$$

1.3.3 Entropi

Gitt en sannsynlighetsfordeling over k klasser, er sannsynligheten for hver klasse p_i . Entropien til fordelingen er da gitt ved:

$$I(p_1, \dots, p_k) = - \sum_{i=1}^k p_i \log_2(p_i)$$

Den maksimale entropien til en binær fordeling er 1, som svarer til lik sannsynlighet per klasse. Den minimale entropien er 0, som svarer til at alle datapunktene tilhører samme klasse.

Før vi splitter på en feature har vi et datasett D med p positive og n negative instanser. Dette svarer til en binær distribusjon der positiv klasse har sannsynlighet $\frac{p}{p+n}$, og negativ klasse har sannsynlighet $\frac{n}{p+n}$, og entropien er

$$I_{\text{før}} = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right). \quad (43)$$

Etter at treet splitter på en feature, har vi to nye datasett, D_1 og D_2 (ett på hver side av splitten). Den forventede entropien etter denne splitten er generelt

$$\mathbb{E}[I_{\text{etter}}] = \sum_{i=1}^s \frac{p_i + n_i}{p+n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right). \quad (44)$$

hvor s angir antall splitt, i vårt tilfelle $s = 2$, slik at $i = 1, 2$. Vi er interessert i forskjellen i entropi, altså entropien før og etter en valgt splitt:

$$\Delta I = I_{\text{før}} - \mathbb{E}[I_{\text{etter}}] = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \sum_{i=1}^s \frac{p_i + n_i}{p+n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right), \quad (45)$$

og vi bør velge den splitten som gir størst endring i entropi. **Oppgave:** Vi har en beslutningsnode bestående av $p = 9$ positive og $n = 5$ negative datapunkter. Vis at $I_{\text{før}} = 0.94$. Anta deretter at vi splitter datasettet slik at vi får to nye datasett med henholdsvis $(p_1 = 4, n_1 = 5)$ og $(p_2 = 5, n_2 = 0)$. Regn ut endringen i entropi (svaret står rett under, men prøv selv).

$$\mathbb{E}[I_{\text{etter}}] = \frac{4+5}{9+5} I\left(\frac{4}{9}, \frac{5}{9}\right) + \frac{5}{9+5} I\left(\frac{5}{5}, \frac{0}{5}\right) = -\frac{9}{14} \left(\frac{4}{9} \log_2 \frac{4}{9} + \frac{5}{9} \log_2 \frac{5}{9} \right) + 0 \approx 0.64. \quad (46)$$

1.3.4 Bygge beslutningstrær

For å bygge beslutningstrær brukes som oftest biblioteket

`sklearn.DecisionTreeClassifier`. Den lar en velge mellom de splitt-kriteriene Gini impurity, entropy og log loss.

Gini impurity og entropy er oppfører seg likt, hovedforskjellen er entropy har en ekstra logaritme i kjøretid. Gini impurity er mindre beregningstungt og default i

`sklearn.DecisionTreeClassifier`.

Utover valg av beslutningskriterier, må vi bestemme:

- Når skal vi slutte å splitte, selv om nederste node har instanser fra begge klassene?
- Hva gjøres med løvnoder med instanser fra begge klassene?
Kan oppstå for 3 tilfeller vi må slutte å splitte:
 1. Når det ikke finnes flere features å splitte på. Dvs har splittet på alle tilgjengelige features, men har ikke laget løvnoder som tilordner alle treningsdatapunktene til riktig klasse.
 2. Når det ikke finnes flere datapunkter å teste. Alle kombinasjoner av features som er **tilgjengelig i dataene** har blitt testet, men alle tenkelige kombinasjoner av features ikke er testet.

3. Når treet har nådd en predifinert maksimal dybde fra hyperparameter vi velger før begynner å bygge treet.

Etter at treet er bygget vil vi sannsynligvis ha løvnoder som inneholder treningsdatapunkter fra begge klasser. For å bestemme hvilken beslutning en slik node kan ta har vi flere muligheter, hvorav de vanligste er å returnere:

1. Den dominante klassen i noden, altså label tilsvarende den dominante klassen fra treningsdataene i løvnoden.
2. Et tilfeldig trukket label fra treningsdataene, altså **a priori**-sannsynligheten.

Vi kan nå sette sammen alt vi har vært gjennom til en algoritme som rekursivt bygger et beslutningstre, se pseudokode under.

```
def build_decision_tree(data, features, depth):
    if (all data in same class):
        return class label
    elif (no features left to test) or (depth >= max_depth):
        return 1 if p/(p+n) > n/(p+n) else 0
    elif (no data left to test):
        return (dominant class in parent node)
    else:
        choose best_feature f for split
        partition data based on f
        for each data partition:
            build_decision_tree(data partition, features - best_feature, depth+1)
```

I forelesningen brukes datasettet Palmer Penguins som eksempel. Dette datasettet er superpopulært innenfor maskinlæring, og kan brukes til mye mer enn å lage et beslutningstre som finner pingvinkjønn basert på kroppsekt og flipper length. Hvis du vil reproducere noen av eksemplene, finner du en .csv-fil på internett som inneholder dataene, og kommer i gang med koden under.

```
df = pd.read_csv("data/penguins_lter.csv")
df = df[["Flipper Length (mm)", "Body Mass (g)", "Sex"]].dropna()
label_encoder = LabelEncoder()
df["Sex"] = label_encoder.fit_transform(df["Sex"])

X_data = df[["Flipper Length (mm)", "Body Mass (g)"]].values
y_data = df['Sex'].values

X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.3)

clf = tree.DecisionTreeClassifier(max_depth=7, criterion="log_loss")
clf.fit(X_train, y_train)
```

Og så er det bare å kose seg. For å visualisere beslutningsflaten slik vi så på i forelesning, kan du bruke `from sklearn.inspection import DecisionBoundaryDisplay`.

1.4 Regresjon

1.4.1 Data og tapsfunksjon

Prediksjon til kontinuerlige verdier kalles **regresjon**. Generelt: estimering av en (eller flere) kontinuerlige verdier. Enkleste tilfellet er lineære regresjonsmodellen:

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_n x_n$$

Kan bruke igjen mye av kode fra klassifisering, og har igjen en bias β_0 og vektor/parameter β_i for hver dataegenskap x_i . Vi bruker igjen `#gradient_descent` for å optimalisere parametrene.

Vanligste tapsmodellen innen regresjon er `#mean_squared_error` :

$$\text{MSE} = \frac{1}{2N} \sum_{i=1}^N [y^{(i)} - f(x^{(i)})]^2$$

Her er $y^{(i)}$ er target for datapunkt $\mathbf{x}^{(i)}$, f er modellen og summen (gjennomsnittet) går over alle N instansene (radene) i datasettet. Liten MSE gir god prediksjon og stor gir dårlig. Gradient descent for et datasett med en feature x_1 gir oss:

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = \frac{1}{N} \sum_{i=1}^N (\beta_1 x^{(i)} + \beta_0 - y^{(i)})$$
$$\frac{\partial \mathcal{L}}{\partial \beta_1} = \frac{1}{N} \sum_{i=1}^N x^{(i)} (\beta_1 x^{(i)} + \beta_0 - y^{(i)})$$

Oppdateringsregel til parametrene i regresjonsmodellen blir da:

$$\beta_0 \leftarrow \beta_0 - \eta \frac{1}{N} \sum_{i=1}^N (f(x^{(i)}) - y^{(i)}) \quad \beta_1 \leftarrow \beta_1 - \eta \frac{1}{N} \sum_{i=1}^N x^{(i)} (f(x^{(i)}) - y^{(i)})$$

1.4.2 Bias og varians

Skille mellom bias leddet β_0 og bias til en modell som forskjellen mellom den sanne verdien vi prøver å estimere, og forventningsverdiene til estimatet verdien:

$$\text{Bias}(y, \hat{y}) = \mathbb{E}[\hat{y} - y] = \mathbb{E}[\hat{y}] - y$$

Der $\hat{y}$ er en estimator (`y_pred`) beregnet på et tilfeldig utvalg datapunkter. Annet nyttig er varians, som gir spredningen prediksjonene har fra gjennomsnittssverdien:

$$\text{Var}[\hat{y}] = \mathbb{E} \left[(\hat{y} - \mathbb{E}[\hat{y}])^2 \right]$$

Dette er nyttig da

- høy varians ofte indikerer at modellen er i overkant sensitiv til variasjoner i data, som tyder på overtilpasning (overfit),
- lav varians ofte indikerer at modellen predikerer for nært gjennomsnittsprediksjonen, og ikke gjør tilstrekkelig nytte av informasjonen i features, som tyder på undertilpasning (underfit).

For en og samme modell finnes det en avveining mellom bias og varians, kjent som `#bias_variance_tradeoff`. Kan utlede for MSE-tapsfunksjonen (se hefte):

$$\text{MSE}[y, \hat{y}] = \text{Bias}(y, \hat{y})^2 + \text{Var}(\hat{y})$$

Dette er viktig for å innse at en modells tap består av en komponent fra bias og en fra varians. Dette skaper "the bias variance dilemma" fordi å minske disse kildene til prediksjonsfeil fører til en konflikt, kan ikke ha begge for lave ettersom det gir en dårlig modell. (Lav bias gir veldig generalisering på treningsdataene slik at ikke bruker features ordentlig, som også går for varians.)

1.4.3 Feature engineering

Annvend lineær regresjonen til formen av testdataene dine! Gjør om featuresene x fra linear til polynom ved $\{x_1, x_2\} = \{x, x^2\}$ og sett opp lineærregresjonen med to features istedet.

NB! funker for alle tilfeller, ikke kun for veiledet læring eller regresjon!

Generelt er feature engineering alle operasjoner vi utfører på datasettet vi bruker til maskinlæring, og inkluderer:

- **Feature selection**, altså utvalgelse av features, som da vi valgte å ikke ta med "Name" i klassifiseringsoppgaven på Titanic-dataene.
- **Feature preprocessing**, altså preprosessering av features, som da vi skalerte "Age" til intervallet (0, 1) for Titanic-dataene.
- **Feature extraction**, altså utvinning av features, som da vi nettopp laget en feature x_2 fra x . Det er også mulig å kombinere flere eksisterende features til nye.

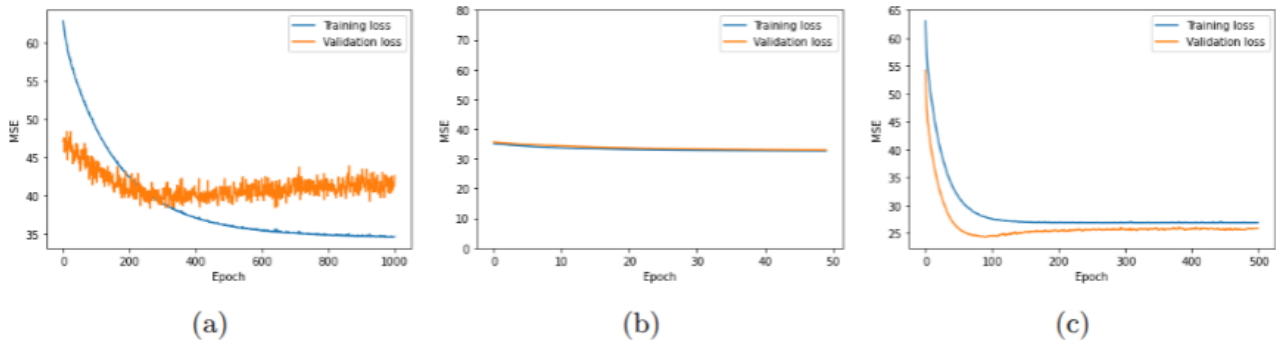
1.4.4 Trening, testing og validering

Overtilpasning skjer når modellen har kapasitet til å tilpasse seg så godt til treningsdataene at det går utover generaliseringsevnen på testdatene. Viktig å kunne detektere overtilpasning slik at vi kan juste hyperparametrene underveis i treningen. Men vi kan ikke bruke trenings- eller testdataene til å tilpasse hyperparametrene, så det er vanlig å dele datasettet i tre deler:

- **Treningsdata** brukes til å tilpasse modellparametrene (trening).
- **Testdata** brukes for å rapportere ytelsen til den endelige modellen, brukes ikke til å gjøre noe som helst tilpasning.
- **Valideringsdata** brukes for å monitorere modellen under trening, for å kunne justere hyperparametrene.

For å ta i bruk valideringsdataen, må treningsprosedyren regne ut relevante metrikker på valideringsdataene et gitt antall ganger i løpet av treningen. Dette kan gjøre f.eks med å beregne tapet for både trenings- og valideringsdata og plote opp mot

hverandre:



a) Overtilpasser, b) undertilpasser, c) optimal tap for test og validering

- Hvis tap på treningsdataene synker jevnt, mens loss pp valideringsdataene er høyere og/eller flater ut, overtilpasser modellen.
- Hvis begge kurvene flater ut med høy loss, undertilpasser modellen. Det betyr som oftest at den ikke har kapasitet eller riktig form til å tilpasse seg til dataene. Det mest åpenbare tegnet på undertilpasning er høy loss på treningsdataene.
- Hvis begge kurvene synker jevnt og flater ut, betyr det at modellen klarer å modellere trenings- dataene, og samtidig klarer å generalisere til nye data. Kan også bruke metrikkene til å justere hyperparametre, som læringsraten eller avgjøre hvor mange epoker en skal bruke for treningen.

1.4.5 Kryssvalidering

Hvordan skal dele inn i trenings-, test- og valideringsdata? Skal gjøres tilfeldig, men det vil påvirke testresultatet, avhengig av datasplitten. En løsning er `#kryssvalidering`. Enkleste formen er **leave-on-out-cross-validation** (`#LOOCV`), hvor en observasjon fjernes fra det opprinnelige datasettet og brukes til testing.

For totalt n datapunkt vil vi kjøre trening på $n - 1$ datapunkt, også loope igjennom for hvert datapunkt og bruke en og en som testdataen. De n resultatene tar vi gjennomsnittet av for å få LOOCV-estimatet.

Annen form er `#k-fold_cross_validation`, med k -splitter istedet for n . Man kjører da samme oppsett, men med $n - k$ treningsdatapunkt og k testdatapunkt.

1.4.6 Regresjonstrær

For tydelige fordelinger av target data med ukjent funksjonsform vil ren regresjon funke dårlig. Istedet kan fordele dataene i flere regioner ved å bruke beslutningstrær, hvor hver splitt i treet deler opp datarommet. Deretter kan en predikere en verdi per område.

Table 7: Forskjeller mellom beslutningstrær og regresjonsmodeller.

Trær	Regresjon
Feature scaling er ikke nødvendig. Beslutningstrær deler inn dataene i regioner, så det er ikke nødvendig å skalere dem.	Features bør ha samme størrelsesorden, helst $\mathcal{O}(1)$, så ikke tilpasningen av modellparametrene fører til store gradienter.
Hvis læringsalgoritmen ikke har begrensninger i tredybde, kan treet bli for dypt og overtilpasse.	Funksjonsformen er bestemt. Læringsalgoritmen kan ikke legge til ledd og lage en mer kompleks modell, eller fjerne ledd om mindre kompleksitet er tilstrekkelig.
Har en overordnet struktur, men den spesifikke oppbygningen – <i>arkitekturen</i> – bestemmes under trening.	Har en gitt likningsform med antall ledd, og parametrene til hver variabel og variabelkombinasjon tilpasses.

En viktig egenskap for regresjonsmodeller og beslutningstrær er at de er tolkbare:

- For regresjonsmodeller, om β_4 er stor, skjønner vi at x_4 har stor betydning på prediksjonen
- For beslutningstrær er splittkriteriene forståelig for mennesker, og features som splittes tidlig er viktigere enn de som splittes senere.

1.5 Ensemble-modeller

Alle modeller har sine antakelser og svakheter. Tanken bak `#ensemble_learning` er at flere modeller kan kombineres, slik at de kompenserer for hverandres svakheter, og til sammen utgjør en samling (ensemble), som benytter seg av hver enkelt modells styrke.

Vi har konseptuelt 3 ulike måter for å kombinere flere modeller:

- **Parallelt:** Flere modeller trenes uavhengig av hverandre, og prediksjonene deres kombineres til en enkelt prediksjon.
- **Sekvensiet:** Flere modeller kommer etter hverandre, og hver modell opphever feilen begått av foregående modell. Til sammen kommer rekken av modeller frem til en prediksjon, der hver modell har minimert feilen gjort av modellen før.
- **Hierarkisk:** Vi bruker en (eller flere) modeller til å kombinere prediksjonen fra en foregående parallellkombinasjon av flere modeller.

Når vi bruker en trent modell til å gjøre prediksjoner, sier vi at vi gjør `#inferens`. Skillet mellom trening og inferens er viktig, særlig synlig i ensemble learning.

Kan lage et parallellt ensemble på flere måter:

- Trene samme type modell med n ulike valg av hyperparametre, for hvert datapunkt vil vi få n ulike prediksjoner som vi **aggregerer** til en endelig prediksjon.
- Trene n ulike type modeller, og aggregere prediksjonene til en endelig prediksjon.
- Kan også trene ulike modellene med ulike deler av treningsdataene.

1.5.1 Bagging

Må ta stilling til to spørsmål ved bruk av ensemble med ulike deler av treningsdataene og kombinere prediksjonene:

1. Hvordan aggregere de ulike modellenes prediksjoner?
2. Hvordan velge ut hvilke deler av treningsdataene hver enkelt modell trener på?

Aggregering:

For regresjon tar en som oftest gjennomsnitt av alle modellenes prediksjoner.

For klassifisering gjøres det som oftest med å velge flertallet av enkeltmodellenes prediksjoner eller å gjøre en vektet avstemning mellom modellene.

Utvalg av treningsdata:

`#bootstrapping` er et sentralt konsept i utvalg av treningsdata. En underliggende tanken bak bootstrapping er at vi vet at vi ikke kan samle nok data til å representere den underliggende fordelingen bak et fenomen perfekt, men gitt et stort nok datasett kan vi få til et tilstrekkelig representativt utvalg. Likevel vil et representativt utvalg ikke uten videre fortelle oss om usikkerheten i estimatene vi gjør basert på disse dataene, altså hvor stor spredning det har. Gitt datasettet vi har samlet kan vi dog lage et estimat av spredning, eller usikkerhet, ved hjelp av teknikken **bootstrapping**.

Dette går ut på å trekke flere datapunkter fra det samme datasettet med tilbake-legging (dette er viktig: det samme datapunktet kan finnes flere ganger i resulterende datasett), og slik ende opp med flere ulike datasett fra det ene datasettet vi startet med. Vi bruker disse ulike datasettene til å estimere den samme størrelsen flere ganger, og slik ende opp med en fordeling av estimatene.

Denne fordelingen kan vi bruke til å beregne en spredning, eller usikkerhet, i estimatet vårt. Vi startet altså med ett datasett som vi kunne lage ett estimat fra, men har ved hjelp av bootstrapping skaffet oss en fordeling – uten å ha fått tilgang til flere datapunkter eller datasett. Vi har laget mer uten å måtte samle mer data, altså “pulled us up by our own bootstraps”.

[BAGGING kommer fra Bootstrap + AGGregerING]

Eksempel på ensemblemodell som bruker bagging er `#random_forest`. Lages ved å sette sammen ulike beslutningstrær, eventuelt stumper. For å få god modell må de ulike trærne være diverse og uavhengige, som gjøres ved å trene på ulike deler av dataene med bootstrapp-teknikken, bruke ulike utvalg data-features og til slutt aggregere prediksjonene sammen – bagging har skjedd.

1.5.2 Boosting

Brukes ofte innen ML for algoritmer som iterativt (sekvensielt) trener svake modeller på en datafordeling, og kombineres til en sterk modell (ensemblen). Går ut på at feilene

begått av en modell gjør den påfølgende modellen i iterasjonen bedre, derved "booster" den. Vi ser på to algoritmer som gjør dette:

AdaBoost-algoritmen bygger et ensemble av modeller som korrigerer hverandres feil gjennom en iterativ treningsprosedyre. I starten av prosedyren har alle punktene i treningsdataene samme vekt. Etter hver modell trenes, ser vi basert på targets hvilke datapunkter som modellen har størst tap, og øker vektene for disse datapunktene.

Gradient-boosting baseres ikke på vekting av observasjoner. Hver modell predikerer forskjellen mellom targets og den forrige modellens prediksjon, såkalte

`#pseudo_residuals`. Det nye ensamlet lages ved å følge læringsregelen

```
new_ensemble = previous_ensemble - learning_rate * new_tree
```

Minner om gradient descent, men gjør gradient descent i rommet over alle mulige trær ensemblet kan bestå av (istedet for i rommet over alle mulige parameterverdier).

Det er en god regel å alltid bruke en ensemble-modell som referanseverdi for hvor godt en modell kan gjøre det, når du jobber med et maskinlæringsproblem med tabulære data.

2 Nevrale nettverk